Buổi 5

Hiệu năng và quản lý bộ nhớ

Đo trước khi sửa · change detection · virtual scroll · heap snapshot

Mục tiêu buổi học

Sau buổi này, học viên có thể:

1. Đo **baseline** lặp lại được: cùng thiết bị, cùng dữ liệu, cùng thao tác, nhiều lần, ghi dữ liệu thô
2. Đọc **Chrome Performance** và **Angular DevTools Profiler** để tìm việc thừa
3. Giảm việc render: `OnPush`/zoneless, `track` theo id, tách phần cập nhật liên tục, `@defer`, lazy loading
4. Chọn giữa **phân trang server** và **virtual scroll** theo dữ liệu và thao tác
5. Tìm rò bộ nhớ bằng **heap snapshot**: so sánh snapshot, đọc retainer path, sửa, đo lại

Sản phẩm đầu ra: bảng benchmark có dữ liệu thô · profile trước và sau · kết luận nêu điều kiện áp dụng

Lịch buổi học (4 giờ)

Thời gian	Hoạt động
20'	Review buổi 4 (ma trận state, ADR-0003), phân tích use case
40'	Quy trình đo, công cụ, change detection, DOM lớn, bộ nhớ
30'	Live demo: Performance panel, DevTools Profiler, heap snapshot trên màn hình Báo cáo
10'	Nghỉ
120'	Lab 5 bài
20'	Review chéo, chốt kết luận benchmark

Use case: màn hình Báo cáo thuê bao

Trưởng nhóm CSKH cần xem 1.000–10.000 thuê bao trên một màn hình để đối chiếu. Điểm bắt đầu `session05-start`:

- `/subscribers/report`: chọn số dòng, bảng HTML thường, đồng hồ **ca trực** cập nhật mỗi giây
- Backend chạy với `MAX_PAGE_SIZE=10000` (mặc định giới hạn 200 để bảo vệ server)

Lời phàn nàn	Đo được (máy M4, 1×)
"Chọn 10.000 dòng phải chờ"	1.181 ms; máy yếu (4×) 3.394 ms
"Bấm Tải lại cũng chờ y như lần đầu"	933–1.104 ms; 4×: 3,7–4,5 giây
"Mở báo cáo vài lần là trình duyệt ì"	Heap +46,5 MB sau 20 lần điều hướng

> Chưa đoán nguyên nhân. Đo trước.

Quy trình đo: một lần đo không phải là số đo

Giữ cố định	Ví dụ
Thiết bị, trình duyệt, phiên bản	Ghi rõ CPU, RAM, Chrome
Điều kiện CPU / mạng	Throttle 4× (máy yếu), Fast 4G
Build	Production nếu đo cho người dùng; ghi rõ nếu là dev
Dữ liệu	Cùng 10.000 dòng, cùng thứ tự
Thao tác	Bắt đầu, kết thúc ở đâu (bấm → vẽ xong)
Số lần	Tối thiểu 3–5, bỏ lần khởi động, ghi **tất cả**

> Đổi hai thứ cùng lúc thì không biết thứ nào có tác dụng. Mỗi lần chỉ đổi một thứ, đo lại.

Đáp án có kịch bản đo lặp lại được: `PERF=1 THROTTLE=4 npx playwright test ... report.perf` (Playwright + Chrome DevTools Protocol).

Công cụ

Công cụ	Trả lời câu hỏi
Chrome **Performance**	Thời gian đi đâu: scripting, rendering, painting; long task nào
Angular DevTools → Profiler	Change detection chạy bao nhiêu lần, component nào tốn
Performance monitor (DevTools)	DOM nodes, JS heap, layout/giây theo thời gian thực
Memory → Heap snapshot	Object nào còn sống, ai giữ nó (retainer)
Tab **Network**	Payload bao nhiêu kB, nén chưa
Console (dev mode)	Cảnh báo của Angular, vd `NG0956`

> Bắt đầu bằng câu hỏi ("vì sao tải lại chậm?"), chọn công cụ trả lời câu hỏi đó. Không mở cả 6 rồi ngắm.

Angular tự cảnh báo bạn

```
NG0956: The configured tracking expression (track by identity) caused re-creation
of the entire collection of size 10000. This is an expensive operation requiring
destruction and subsequent creation of DOM nodes, directives, components etc.
```

```
<!-- Điểm bắt đầu -->
@for (s of rows(); track s; let i = $index) { <tr>...</tr> }

<!-- Mỗi lần tải lại, server trả object MỚI → khác identity → hủy và dựng lại 10.000 <tr> -->
@for (s of rows(); track s.id; let i = $index) { <tr>...</tr> }
```

Chỉ hiện ở dev mode. Đọc console khi phát triển là cách rẻ nhất để bắt lỗi hiệu năng.

Change detection trong ứng dụng zoneless

- Care Portal **zoneless** + mọi component `OnPush`: Angular chỉ kiểm tra view khi signal nó đọc đổi, input đổi, hoặc có sự kiện trong view
- Nhưng **kiểm tra một view = đánh giá lại mọi binding trong template của view đó**

```
// Điểm bắt đầu: đồng hồ là signal TRONG trang báo cáo
protected readonly now = signal(new Date());
constructor() { setInterval(() => this.now.set(new Date()), 1000); }
```

```
<span>Ca trực: {{ now() | translocoDate: timeFormat }}</span>
...
@for (s of rows(); track s) {
  <td>{{ ageOf(s.dateOfBirth) }}</td>              <!-- 10.000 lần gọi mỗi giây -->
  <td>{{ activatedOn(s) }}</td>                    <!-- 10.000 lần toLocaleDateString mỗi giây -->
```

Mỗi giây: `now` đổi → kiểm tra cả trang → 30.000 lần gọi method. Ngồi yên 5 giây tốn 150 ms CPU (247 ms trên máy yếu).

Tách phần cập nhật liên tục, tính sẵn dữ liệu

```
@Component({ selector: 'care-shift-clock',
  template: `{{ 'report.shift' | transloco }}: {{ now() | translocoDate: timeFormat }}` })
export class ShiftClock {
  protected readonly now = toSignal(interval(1000).pipe(map(() => new Date())),
    { initialValue: new Date() });                    // tự hủy interval khi component bị hủy
}
```

```
// View model tính MỘT lần khi dữ liệu đổi, không phải mỗi lần kiểm tra view
protected readonly rows = computed(() => {
  const today = new Date();
  return this.result.hasValue() ? this.result.value().map((s) => toReportRow(s, today)) : [];
});
```

```
<td>{{ s.age }}</td>  <td>{{ s.statusKey | transloco }}</td>  <td>{{ s.activatedAt | translocoDate: dateFormat }}</td>
```

Cũng sửa một lỗi i18n: `toLocaleDateString('vi-VN')` cứng → chế độ EN vẫn hiện ngày kiểu Việt.

Angular DevTools Profiler: đọc gì

- **Record**, ngồi yên 3 giây, **Stop**: mỗi cột là một lần change detection
- Trước: mỗi giây một cột cao, component `SubscriberReportPage` chiếm gần hết thời gian
- Sau khi tách đồng hồ: mỗi giây chỉ `ShiftClock` được kiểm tra, cột gần như phẳng

Câu hỏi	Nhìn ở đâu
CD chạy bao nhiêu lần?	Số cột trong thanh thời gian
Component nào tốn?	Bar chart / flame graph của một lần CD
Vì sao chạy?	Nguồn kích hoạt (signal, sự kiện, input)

> Profile trước và sau là **sản phẩm nộp**: chụp màn hình cả hai, ghi điều kiện đo.

DOM lớn: ba lựa chọn

	Phân trang server	Tải hết, DOM thường	Tải hết, virtual scroll
Payload	Vài chục kB / trang	2,93 MB (10.000 dòng)	2,93 MB
Node DOM	~vài trăm	**100.167**	**357**
Tải lại (1×, đo được)	~vài chục ms	933–1.104 ms	~150 ms
`Ctrl+F` trình duyệt	Chỉ trang hiện tại	Có	Chỉ dòng đang vẽ
In / chọn hết	Không	Có	Không (cần xuất file)
Hợp với	Tìm kiếm, tra cứu (trang Thuê bao)	Ít dòng (< vài trăm)	Cuộn xem nhanh nghìn dòng, đã có hết dữ liệu

> Mặc định: phân trang server. Virtual scroll khi người dùng thật sự cần cuộn qua nhiều dòng và đã có hết dữ liệu ở client.

Virtual scroll với CDK

```
<div class="panel report" role="table" [attr.aria-rowcount]="rows().length + 1">
  <div class="report-row report-head" role="row" aria-rowindex="1">
    <span role="columnheader">#</span> ...
  </div>
  <cdk-virtual-scroll-viewport class="report-viewport" [itemSize]="rowHeight">
    <div class="report-row" role="row"
         *cdkVirtualFor="let s of rows(); let i = index; trackBy: trackById"
         [attr.aria-rowindex]="i + 2">
      <span role="cell">{{ s.msisdn }}</span> ...
    </div>
  </cdk-virtual-scroll-viewport>
</div>
```

- Mọi dòng **cùng chiều cao** (`itemSize` khớp CSS): CDK tính vị trí mà không đo DOM
- `<table>` không hợp với viewport cuộn → lưới CSS + `role="table/row/cell"` + `aria-rowcount` / `aria-rowindex` để trình đọc màn hình vẫn biết có 10.001 hàng

`@defer` và lazy loading

```
<!-- Minh họa (chưa có trong repo): tách lịch sử phiếu ở cuối trang chi tiết thuê bao thành component riêng -->
@defer (on viewport) {
  <care-ticket-history [subscriberId]="s.id" />
} @placeholder {
  <div class="panel detail-panel muted">{{ 'subscriber.detail.recentTickets' | transloco }}</div>
}
```

Kỹ thuật	Tách gì	Khi nào tải
`loadComponent` / `loadChildren` (buổi 1)	Code theo route	Khi vào route
`@defer (on viewport / idle / interaction)`	Code + render một khối trong trang	Khi khối vào màn hình / rảnh / người dùng tương tác

Bundle ban đầu ~1,29 MB, riêng CSS ng-zorro ~577 kB: đặt budget và kiểm tra trên CI ở buổi 9.

Bộ nhớ: rò ở đâu?

```
// Điểm bắt đầu
constructor() {
  setInterval(() => this.now.set(new Date()), 1000);              // không clearInterval
  fromEvent(window, 'resize').pipe(debounceTime(200))
    .subscribe(() => this.compact.set(window.innerWidth < 1200)); // không hủy
}
```

```
Window ──listeners──► resize handler (closure) ──► SubscriberReportPage ──► rows ──► 10.000 object
                                                                          └──► view ──► <tr> đã tách khỏi DOM
```

- Rời trang: Angular hủy component, nhưng `window` còn giữ handler → giữ cả trang và dữ liệu
- Mỗi lần mở báo cáo thêm một bản. Đo được: **+46,5 MB** sau 20 lần (bảng DOM), **+11 MB** (virtual scroll)

Heap snapshot: quy trình 3 snapshot

1. Mở trang Danh sách, **GC** (biểu tượng thùng rác), **Snapshot 1**
2. Báo cáo ↔ Danh sách **20 lần** (điều hướng trong app, không F5), GC, **Snapshot 2**
3. Thêm 20 lần nữa, GC, **Snapshot 3**
4. Snapshot 3, chế độ **Comparison** với Snapshot 2: lọc `SubscriberReportPage`, `Detached`
5. Chọn một object → panel **Retainers**: đọc đường giữ ngược về `Window`

Dấu hiệu	Kết luận
`# New` của `SubscriberReportPage` = 20	Mỗi lần điều hướng giữ lại một trang
`Detached HTMLTableRowElement` hàng nghìn	DOM đã gỡ vẫn bị giữ
Heap tăng sau một lần, không tăng tiếp	Cache/khởi tạo, **không phải rò**

Sửa rò: gắn mọi subscription với vòng đời

```
protected readonly compact = toSignal(
  fromEvent(window, 'resize').pipe(debounceTime(200), map(() => window.innerWidth < 1200)),
  { initialValue: window.innerWidth < 1200 },
);
```

```
it('rời màn hình: gỡ listener resize', async () => {
  const remove = vi.spyOn(window, 'removeEventListener');
  const fixture = TestBed.createComponent(SubscriberReportPage);
  await fixture.whenStable();
  fixture.destroy();
  expect(remove).toHaveBeenCalledWith('resize', expect.any(Function), undefined);
});
```

Đo lại: heap **24,7 → 26,7 MB** sau 20 lần (+2 MB: nhiễu, cache).

Kết quả đo (máy giảng viên, dev build)

Phiên bản	CPU	Lần đầu (ms)	Tải lại (ms)	Node DOM	Ngồi yên 5s (ms)	Heap +20 lần
Điểm bắt đầu	1×	1.181	933–1.104	100.167	150	+46,5 MB
Điểm bắt đầu	4×	3.394	3.746–4.467	100.167	247	+46,5 MB
+ virtual scroll	4×	473	138–166	357	9	+11 MB
+ tách đồng hồ, view model, `track` id	4×	490	150–167	357	4	+11 MB
+ gỡ listener	4×	458	150–167	357	4	+2 MB
Đối chứng: DOM thường + chỉ bài 3	4×	9.213	3.518–3.563	100.167	6	+53,8 MB

Dữ liệu thô đầy đủ (cả 1×) và điều kiện đo: `docs/benchmark-buoi-05.md`.

Kết luận phải có điều kiện

1. **Virtual scroll** là thay đổi lớn nhất **khi** người dùng cần cuộn qua nghìn dòng đã có ở client. Không thì phân trang server rẻ hơn
2. **Tách phần cập nhật liên tục** giảm CPU ngồi yên 150 → 9 ms **khi** DOM lớn; sau virtual scroll chỉ còn 6 → 4 ms
3. `track` **theo id một mình không làm tải lại nhanh hơn** khi vẫn vẽ 10.000 dòng: chi phí chính là kiểm tra lại binding và pipe, không phải dựng DOM. Vẫn sửa vì đúng (và Angular cảnh báo)
4. **Pipe định dạng theo từng dòng có giá:** vẽ đủ 10.000 dòng với `translocoDate` chậm hơn ở lần đầu (2.780 so với 1.181 ms). Phải vẽ đủ → tính sẵn chuỗi theo ngôn ngữ
5. **Rò bộ nhớ** không làm chậm ngay, nhưng tích lũy theo số lần điều hướng: CSKH mở báo cáo cả ngày

> Kết luận "nhanh hơn" không có điều kiện là kết luận sai. Ghi rõ: dữ liệu bao nhiêu, thiết bị nào, thao tác gì.

Live demo (30 phút)

```
MAX_PAGE_SIZE=10000 docker compose -f infra/docker-compose.yml up -d --build api
```

1. **Console:** chọn 10.000 dòng, bấm Tải lại → `NG0956`
2. **Performance** (CPU 4×): record "Tải lại" → long task vài giây, phần lớn là scripting + rendering
3. **Angular DevTools Profiler:** ngồi yên 3 giây → mỗi giây một lần CD toàn trang
4. **Memory:** quy trình 3 snapshot → `SubscriberReportPage` ×20, retainer path về `Window`
5. Chuyển sang `session05-end`, làm lại 2–4

Lab (120 phút)

Bài	Nội dung	Thời gian
1	Baseline: quy trình đo, ghi dữ liệu thô (Performance, Performance monitor, Network)	25'
2	Virtual scroll bằng CDK; so với DOM thường và phân trang server	35'
3	Change detection: Profiler, tách đồng hồ, view model, `track` id	25'
4	Heap sau 20 lần điều hướng: tìm retainer, sửa, đo lại	25'
5	Hoàn thiện `docs/benchmark-buoi-05.md`: dữ liệu thô, profile trước/sau, kết luận có điều kiện	10'

Điểm bắt đầu: `git checkout session05-start` · Đề bài: `course/session-05/lab/README.md`

Bẫy hay gặp khi làm lab

Hiện tượng	Nguyên nhân
Chọn 10.000 dòng báo lỗi	Backend chưa chạy với `MAX_PAGE_SIZE=10000`
Số đo mỗi lần một khác	Đang mở DevTools Profiler khi đo Performance; tab khác chạy nền; không throttle cố định
Virtual scroll trắng hoặc nhảy	`itemSize` không khớp chiều cao CSS; viewport không có chiều cao
Bảng virtual scroll mất cột tiêu đề	Đặt tiêu đề trong viewport (cuộn mất)
Heap không giảm sau khi sửa	Chưa bấm GC trước snapshot; còn `setInterval` của đồng hồ
Heap snapshot không thấy rò	F5 giữa các lần điều hướng (tải lại trang xóa hết)
Test `removeEventListener` không được gọi	Vẫn `subscribe()` trần

Tổng kết

1. **Đo trước, sửa sau**: điều kiện cố định, lặp lại, ghi dữ liệu thô, mỗi lần đổi một thứ
2. Zoneless + `OnPush` chưa đủ: **kiểm tra một view là đánh giá lại mọi binding** → tách phần cập nhật liên tục, tính sẵn dữ liệu
3. `track` theo id, đọc cảnh báo `NG0956`
4. DOM lớn: phân trang server mặc định; **virtual scroll** khi cần cuộn qua nhiều dòng đã có ở client
5. Rò bộ nhớ: **heap snapshot so sánh**, đọc retainer; mọi subscription gắn vòng đời (`toSignal`, `takeUntilDestroyed`)
6. **Kết luận phải nêu điều kiện**

Buổi 6: Authentication và bảo mật giao tiếp API: 401/403/409, refresh dùng chung, token reuse.